

Proyecto MVC Docker

vamos a hacer el paso a paso del proyecto MVC del seminario de actualización etapa 1 (MVC con Docker)

Voy a armarlo como guía de clase desde cero: objetivo, estructura del proyecto, código completo, armado del WAR, Dockerfile, ejecución y pruebas. Asumo **Servlet + JSP + Tomcat 10.1 + JDK 17 + Maven**, que es lo que veníamos usando.

Perfecto. **Etapla 1: Proyecto MVC monolítico con Docker.**

Proyecto MVC tradicional con Servlet + JSP + Docker

1. Objetivo de la etapa

Construir una aplicación web Java tradicional, donde:

- El navegador pide una página.
- El servidor Tomcat recibe la petición.
- Un `Servlet` actúa como controlador.
- El controlador carga datos.
- El controlador envía esos datos a una página `JSP`.
- La `JSP` genera el HTML final.
- Toda la aplicación se empaqueta como archivo `.war`.
- El `.war` se ejecuta dentro de un contenedor Docker con Tomcat.

La arquitectura será:

```

Navegador
|
v
Tomcat en Docker
|
v
Servlet Controller
|
v
Modelo Alumno
|
v
JSP View
```

2. Crear el proyecto Maven

Crear una carpeta:

</> Bash

```
mvc-docker-seminario
```

Entrar a la carpeta:

</> Bash

```
cd mvc-docker-seminario
```

Estructura final esperada:

```
mvc-docker-seminario
|
├─ Dockerfile
├─ pom.xml
├─ src
│   └─ main
│       ├── java
│       │   └─ com
│       │       └─ seminario
│       │           ├── modelo
│       │           │   └─ Alumno.java
│       │           └─ servlet
│       │               └─ AlumnoServlet.java
│       └─ webapp
│           ├── index.jsp
│           └─ alumnos.jsp
```

3. Archivo pom.xml

Crear el archivo pom.xml :

</> XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.seminario</groupId>
<artifactId>mvc-docker-seminario</artifactId>
<version>1.0.0</version>
<packaging>war</packaging>

<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <failOnMissingWebXml>false</failOnMissingWebXml>
</properties>

<dependencies>

  <!-- API de Servlet para Tomcat 10 -->
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.0.0</version>
    <scope>provided</scope>
  </dependency>

</dependencies>

<build>
  <finalName>mvc-docker-seminario</finalName>
</build>

</project>
```

4. Crear el modelo Alumno

Ruta:

```
src/main/java/com/seminario/modelo/Alumno.java
```

Código:

Java

```
package com.seminario.modelo;

public class Alumno {

    private int id;
    private String nombre;
    private String carrera;
```

```
public Alumno() {  
}  
  
public Alumno(int id, String nombre, String carrera) {  
    this.id = id;  
    this.nombre = nombre;  
    this.carrera = carrera;  
}  
  
public int getId() {  
    return id;  
}  
  
public String getNombre() {  
    return nombre;  
}  
  
public String getCarrera() {  
    return carrera;  
}  
}
```

5. Crear el controlador AlumnoServlet

Ruta:

```
src/main/java/com/seminario/servlet/AlumnoServlet.java
```

Código:

```
<> Java  
  
package com.seminario.servlet;  
  
import com.seminario.modelo.Alumno;  
import jakarta.servlet.RequestDispatcher;  
import jakarta.servlet.ServletException;  
import jakarta.servlet.annotation.WebServlet;  
import jakarta.servlet.http.HttpServlet;  
import jakarta.servlet.http.HttpServletRequest;  
import jakarta.servlet.http.HttpServletResponse;  
  
import java.io.IOException;  
import java.util.ArrayList;  
import java.util.List;  
  
@WebServlet("/alumnos")  
public class AlumnoServlet extends HttpServlet {
```

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    List<Alumno> alumnos = new ArrayList<>();

    alumnos.add(new Alumno(1, "Ana Gómez", "Programación"));
    alumnos.add(new Alumno(2, "Juan Pérez", "Sistemas"));
    alumnos.add(new Alumno(3, "Lucía Fernández", "Desarrollo Web"));

    request.setAttribute("listaAlumnos", alumnos);

    RequestDispatcher dispatcher = request.getRequestDispatcher("alumnos.jsp");
    dispatcher.forward(request, response);
}
}
```

6. Crear la página inicial index.jsp

Ruta:

src/main/webapp/index.jsp

Código:

 JSP

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Proyecto MVC con Docker</title>
</head>
<body>

    <h1>Proyecto MVC tradicional</h1>

    <p>Aplicación Java Web con Servlet, JSP, Tomcat y Docker.</p>

    <a href="alumnos">Ver listado de alumnos</a>

</body>
</html>
```

7. Crear la vista `alumnos.jsp`

Ruta:

```
src/main/webapp/alumnos.jsp
```

Código:

 JSP

```
<%@ page import="java.util.List" %>
<%@ page import="com.seminario.modelo.Alumno" %>

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Listado de alumnos</title>
</head>
<body>

    <h1>Listado de alumnos</h1>

    <table border="1" cellpadding="8">
        <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Carrera</th>
```

8. Probar localmente con Maven

Desde la carpeta del proyecto:

 Bash

```
mvn clean package
```

Debe generarse:

```
target/mvc-docker-seminario.war
```

9. Crear el `Dockerfile`

En la raíz del proyecto:

Dockerfile

Código:

</> dockerfile

FROM tomcat:10.1-jdk17

COPY target/mvc-docker-seminario.war /usr/local/tomcat/webapps/mvc-docker-seminario.war

EXPOSE 8080

CMD ["catalina.sh", "run"]

10. Construir la imagen Docker

Primero generar el WAR:

</> Bash

mvn clean package

Luego construir la imagen:

</> Bash

docker build -t mvc-docker-seminario .

11. Ejecutar el contenedor

</> Bash

docker run --rm -p 8080:8080 mvc-docker-seminario

12. Probar en el navegador

Abrir:

http://localhost:8080/mvc-docker-seminario/

Luego hacer clic en:

Ver listado de alumnos

O entrar directamente a:

<http://localhost:8080/mvc-docker-seminario/alumnos>

13. Qué deben observar los alumnos

Esta aplicación es **MVC tradicional** porque:

```
Modelo:  
Alumno.java  
  
Vista:  
index.jsp  
alumnos.jsp  
  
Controlador:  
AlumnoServlet.java
```

El servidor genera el HTML final usando JSP.

El navegador no consume una API REST.

El frontend no está separado.

Todo viaja como una única aplicación `.war`.

14. Comandos finales resumidos

</> Bash

```
mvn clean package
```

</> Bash

```
docker build -t mvc-docker-seminario .
```

</> Bash

```
docker run --rm -p 8080:8080 mvc-docker-seminario
```

URL:

<http://localhost:8080/mvc-docker-seminario/>

Esta sería la **Etapla 1 monolítica MVC con Docker**.

